

Prisma kézikönyv (magyar)

Ez a kézikönyv a Prisma ORM részletes, gyakorlati útmutatója TypeScript/Node.js környezetben. Tartalmaz példákat schema kezelésre, migrációkra, Prisma Client használatra, relációkra, tranzakciókra, seed-ekre, deploy és CI tippekre, valamint Prisma integrációra NestJS-szel.

Tartalomjegyzék

1. Összefoglaló — Mi a Prisma?
2. Getting started — Telepítés és init
3. Prisma schema — modellek, mezők, típusok
4. Migrations — prisma migrate használata
5. Prisma Client — generálás és használat TypeScript-ben
6. CRUD példák
7. Relációk és lekérdezési minták (1:1, 1:N, N:N, include/select)
8. Transactions és atomic műveletek
9. Indexek, unique constraint-ek, relation modes
10. Prisma Studio, introspect, db push
11. Seed adatok és tesztelés
12. Deploy és CI/CD tippek
13. Performance és hibakeresés (N+1, connection pooling)
14. Security és best practices
15. Prisma + NestJS integráció példa
16. Prisma + GraphQL / REST példák
17. Docker Compose, sample schema és gyakori hibák

1) Összefoglaló — Mi a Prisma?

- A Prisma egy modern ORM és adatbázis eszközkészlet Node.js/TypeScript környezethez. Fő elemei: Prisma schema (schema.prisma), Prisma Client (automatikusan generált, típusbiztos JS/TS kliens), Prisma Migrate (migrációk kezelése) és Prisma Studio (grafikus DB böngésző).
- Előnyök: típusbiztonság TypeScript-ben, tiszta schema definíció, egyszerű migrációs munkafolyamat, jó DX (developer experience).
- Mire jó: CRUD műveletek, komplex relációk kezelése, gyors prototípus és termelési alkalmazások.

2) Getting started — Telepítés és init

1. Telepítsd a Prisma CLI-t és a kliens könyvtárat:

```
npm install prisma --save-dev
npm install @prisma/client
npx prisma init
```

2. A `npx prisma init` létrehoz egy `prisma/schema.prisma` fájlt és egy `.env` fájlt, amelyben a `DATABASE_URL`-t állíthatod be.

3. Példa `.env`:

```
DATABASE_URL="postgresql://user:pass@localhost:5432/mydb?schema=public"
```

3) Prisma schema — modellek, mezők, típusok

Példa schema (Postgres):

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          Int      @id @default(autoincrement())
  email       String   @unique
  name        String?
  posts       Post[]
  profile     Profile?
  createdAt   DateTime @default(now())
}

model Post {
  id          Int      @id @default(autoincrement())
  title       String
  content     String?
  published   Boolean  @default(false)
  authorId    Int
  author      User     @relation(fields: [authorId], references: [id])
}

model Profile {
  id          Int      @id @default(autoincrement())
```

```

    bio    String?
    userId Int    @unique
    user    User    @relation(fields: [userId], references: [id])
  }

```

Megjegyzések: - `@id`, `@default`, `@unique` és `@relation` direktívák fontosak. - Prisma használ több DB providert: postgresql, mysql, sqlite, sqlserver, mongodb (preview).

4) Migrations — prisma migrate

1. Migrate init és generálás:

```
npx prisma migrate dev --name init
```

- A `migrate dev` fejlesztés közbeni migrációkhoz használatos: módosítja az adatbázist és létrehozza a migration fájlokat.

2. Termelési migrációk:

```
npx prisma migrate deploy
```

- CI/CD pipeline-ban használd, amikor az alkalmazás telepítésekor szeretnéd futtatni a migrációkat.

3. Preview: `prisma migrate reset` (fejlesztéshez: törli adatokat, futtat migrációkat újra).
-

5) Prisma Client — generálás és használat TypeScript-ben

1. Generálás (általában `prisma generate` fut a migrate után):

```
npx prisma generate
```

2. Használat:

```

// src/prisma/client.ts
import { PrismaClient } from '@prisma/client';

export const prisma = new PrismaClient();

```

3. Példa, hogyan használjuk egy service-ben:

```

// src/user.service.ts
import { prisma } from '../prisma/client';

```

```

export async function createUser(data: { name?: string; email: string }) {
  return prisma.user.create({ data });
}

export async function getUsers() {
  return prisma.user.findMany();
}

```

Megjegyzés: PrismaClient példányt általában singleton-ként kell kezelni (különösen dev hot-reload környezetben Next.js/Nodemon esetén ügyelj a lifecycle-re).

6) CRUD példák

Create:

```

const user = await prisma.user.create({
  data: { email: 'alice@example.com', name: 'Alice' },
});

```

Read:

```

const users = await prisma.user.findMany();
const single = await prisma.user.findUnique({ where: { id: 1 } });

```

Update:

```

const updated = await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Alice Updated' },
});

```

Delete:

```

const deleted = await prisma.user.delete({ where: { id: 1 } });

```

7) Relációk és lekérdezési minták

Include és select:

```

// include kapcsolódó posztokat
const userWithPosts = await prisma.user.findUnique({
  where: { id: 1 },
  include: { posts: true },
});

```

```
// select szelektív mezők
const userNameOnly = await prisma.user.findUnique({
  where: { id: 1 },
  select: { id: true, name: true },
});
```

1:N reláció:

```
// létrehozás kapcsolt modellel
const post = await prisma.post.create({
  data: {
    title: 'Hello',
    author: { connect: { id: userId } },
  },
});
```

N:N reláció (példa tags/courses modellek esetén): Prisma generálja a join táblát automatikusan, vagy explicit `@@map/@relation` használható.

8) Transactions és atomic műveletek

Prisma két fő tranzakciós API-t kínál:

- 1) `prisma.$transaction([...queries])` — több lekérdezés egy tranzakcióban:

```
await prisma.$transaction([
  prisma.user.create({ data: { email: 'bob@example.com' } }),
  prisma.post.create({ data: { title: 'tx post', authorId: 1 } }),
]);
```

- 2) Interactive transactions (callback interface) — ajánlott komplex logikához:

```
await prisma.$transaction(async (prismaTx) => {
  const user = await prismaTx.user.create({ data: { email: 'x@y.com' } });
  await prismaTx.post.create({ data: { title: 't', authorId: user.id } });
});
```

Megjegyzés: Ha egy tranzakció több DB kapcsolaton fut (sharding), különös figyelem.

9) Indexek, unique constraint-ek, relation modes

- `@unique` mező szinten biztosítja az egyediségét.
- Kompozit unique és index:

```

model Example {
  a Int
  b Int
  @@unique([a, b])
  @@index([b])
}

```

- `relationMode = "prisma"` vs `"foreignKeys"` (Postgres különböző beállításai). Alap esetben Prisma kezeli a relációk logikáját.

10) Prisma Studio, introspect, db push

- Prisma Studio indítása (fejlesztés):

```
npx prisma studio
```

- Adatbázis introspect:

```
npx prisma db pull
```

- `prisma db push` — a schema alapján az adatbázis sémáját tükrözi (nem hoz létre migrációs fájlokat; dev-ben hasznos gyors prototípusra).

11) Seed adatok és tesztelés

Seed script (package.json):

```

{
  "prisma": {
    "seed": "ts-node prisma/seed.ts"
  }
}

```

prisma/seed.ts vázlat:

```

import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

async function main() {
  await prisma.user.create({
    data: { email: 'seed@example.com', name: 'Seed' },
  });
}

main()

```

```
.catch(e => console.error(e))  
.finally(async () => { await prisma.$disconnect(); });
```

Tesztelés: - Teszt környezethez használj külön adatbázist (CI-ben ephemeral DB).
- Mockolás helyett inkább integration teszteket futtass valódi DB-vel (SQLite in-memory vagy Dockerized Postgres).

12) Deploy és CI/CD tippek

- CI pipeline:
 1. build
 2. `npx prisma migrate deploy` (termelési migrációk)
 3. `npm run start`
 - Használj environment-specifikus `DATABASE_URL`-t.
 - Biztonság: soha ne tárold a DB jelszavát a forráskódban, használj secret manager-t.
-

13) Performance és hibakeresés

N+1 probléma: - GraphQL + Prisma esetén gyakori. Megoldás: DataLoader vagy optimalizált include/select stratégiák.

Connection pooling: - Prisma Client Pooling használata a DB driver rétegével (pl. PgBouncer Postgres esetén). - Figyeld a max connections limitet termelésben.

Lekérdezés optimalizálás: - Csak a szükséges mezőket kérd (`select`), használd az `include`-t tudatosan. - Használj indexeket gyakran szűrt mezőkhöz.

14) Security és best practices

- Least privilege: alkalmazás DB user legyen limitált jogosultságokkal (csak szükséges sémák/olvasás-írás).
 - Secrets: használj Vault/AWS Secrets Manager/GitHub Secrets.
 - Input validation: Prisma önmagában nem validál minden logikát — használj szerver oldali validációt (class-validator stb.).
 - SQL injection: Prisma kliens biztonságos paraméterezésen alapul, de ügyelj raw query-k használatakor.
-

15) Prisma + NestJS integráció példa

- Telepítés:

```
npm i @prisma/client prisma
npm i -D prisma
```

- PrismaService példány (singleton, lifecycle kezeléssel):

```
// src/prisma/prisma.service.ts
import { Injectable, OnModuleInit, OnModuleDestroy } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit, OnModuleDestroy {
  async onModuleInit() {
    await this.$connect();
  }
  async onModuleDestroy() {
    await this.$disconnect();
  }

  // Optional: add enableShutdownHooks for Nest
  async enableShutdownHooks(app: any) {
    this.$on('beforeExit', async () => {
      await app.close();
    });
  }
}
```

- PrismaModule:

```
// src/prisma/prisma.module.ts
import { Global, Module } from '@nestjs/common';
import { PrismaService } from './prisma.service';

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

- Használat egy service-ben:

```
@Injectable()
export class UsersService {
  constructor(private prisma: PrismaService) {}
```



```

    findAll() {
      return this.prisma.user.findMany();
    }
  }
}

```

16) Prisma + GraphQL / REST minták

GraphQL (resolver-ben):

```

@Resolver(() => UserType)
export class UsersResolver {
  constructor(private prisma: PrismaService) {}

  @Query(() => [UserType])
  users() {
    return this.prisma.user.findMany({ include: { posts: true }});
  }
}

```

REST (controller/service):

```

@Get()
async getUsers() {
  return this.usersService.findAll();
}

```

17) Docker Compose példa (Postgres + Prisma)

```

version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - dbdata:/var/lib/postgresql/data

  app:
    build: .

```

```
environment:
  DATABASE_URL: "postgresql://user:pass@db:5432/mydb?schema=public"
depends_on:
  - db

volumes:
  dbdata:
```

Gyakori hibák és megoldások

- “PrismaClientKnownRequestError” — ellenőrizd constraint-eket és migrációkat.
 - Hot-reload alatt többszörös PrismaClient példány — singleton kezelése szükséges (pl. egyetlen PrismaService).
 - Migrációs konfliktusok több fejlesztőnél — merge előtt futtasd a migrációk tesztjét.
-

Források és további olvasmányok

- Prisma docs: <https://www.prisma.io/docs>
 - Prisma GitHub repo: <https://github.com/prisma/prisma>
-

Bővített magyarázatok és minden idegen kifejezés magyarázata

Ez a kiterjesztett fejezet bővíti a korábbi Prisma kézikönyvet: részletesen ismertetem a Prisma kulcskonceptióit, minden fontos annotációt, parancsot, API-t és tipikus hibát, valamint külön, áttekinthető szószeretet (glossary), amelyben minden angol/idegen kifejezést magyarul is elmagyarázok.

A cél: hogy ne csak “hogyan”, hanem a “miért” is világos legyen — amikor idegen szó vagy rövidítés szerepel, rögtön megtalálod a magyarázatát a Glossary-ben.

Tartalom 1. Rövid áttekintés és cél 2. Prisma elemei részletesen: generator, datasource, model, field attribútumok 3. Tipikus schema minták és relációk (1:1, 1:N, N:N), kapcsolat módok és referenciális akciók 4. Migrations részletesen: migrate dev, deploy, resolve, drift 5. Prisma Client API alaposan: findUnique,

findFirst, findMany, create, update, upsert, delete, aggregate, groupBy 6. Transakciók és atomikus műveletek: \$transaction, interactive transactions 7. Indexek, constraint-ek, composite keys és performance tippek 8. Prisma Studio, db pull (introspect), db push — mikor mit használjunk 9. Seedelés, tesztelés, ephemeral DB-k CI-ben, sqlite vs Postgres tesztek 10. Hibák és kivételkezelés: PrismaClientKnownRequestError és társai — mi mit jelent és hogyan kezeljük 11. Security, connection strings, pooling, PgBouncer, connection limits 12. Prisma Data Proxy (ha elérhető) és mikor érdemes használni 13. NestJS + Prisma: tipikus integrációs minták (PrismaService, request-scoped problémák) 14. Best practices checklist 15. Glossary — minden idegen kifejezés magyar magyarázata

1) Rövid áttekintés és cél

- Mi a Prisma röviden?
 - A Prisma egy modern, TypeScript-barát “ORM” és adatbázis eszközkészlet Node.js alkalmazásokhoz. Az “ORM” kifejezés (Object-Relational Mapping) jelentését lásd a Glossary-ben.
- Mit ad a fejlesztőnek?
 - Típusbiztos kliens API-t (Prisma Client), egy deklaratív sémát (schema.prisma), migrációs eszközt (Prisma Migrate), valamint grafikus böngészőt (Prisma Studio).

2) Prisma elemei részletesen

generator - A generator definíció mondja meg, hogy milyen kliens (vagy más eszköz) generálódjon a sémából.

```
generator client {  
  provider = "prisma-client-js"  
  output   = "../src/generated/prisma-client"  
}
```

- provider: mely kódgenerátort használjuk (jellemzően “prisma-client-js”).
- output: opcionális útvonal, hova generálódjon.

datasource - Itt határozod meg az adatforrást (pl. PostgreSQL).

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

- provider: adatbázis típusa (postgresql, mysql, sqlite, sqlserver, mongodb).
- url: connection string (csatlakozási karakterlánc) — környezeti változó javasolt.

model - A model a táblát/gyűjteményt reprezentálja. Mezők (fields) attribútumokkal bővíthetők.

```

model User {
  id Int @id @default(autoincrement())
  email String @unique
  posts Post[]
}

```

Field attribútumok (gyakori) - @id — primer kulcs. - @default(...) — alapértelmezett érték (pl. autoincrement(), now()). - @unique — egyedi index. - @relation(fields: [...], references: [...]) — explicit relációt definiál. - @map("db_column") — adatbázisbeli oszlopnév map-elése. - @@index([...]) és @@unique([...]) — model szintű indexek és kompozit unique-ok. - @@map("table_name") — tábla átkötése más névre a DB-ben.

Példa — kompozit kulcs:

```

model Membership {
  userId Int
  groupId Int
  role String

  @@id([userId, groupId])
}

```

3) Relációk és referenciális akciók

Reláció típusok - 1:1 — egy usernek egy profile-ja lehet: `prisma model User { id Int @id @default(autoincrement()) profile Profile? } model Profile { id Int @id @default(autoincrement()) }`
 - 1:N — egy user több postot írhat: `prisma model User { posts Post[] } model Post { author User @relation(fields: [authorId], references: [id]); authorId Int }`
 - N:N — Prisma automatikusan kezeli a join táblát, ha két modell tömbbel hivatkozik a másikra.

Referential actions (referenciális akciók) - onDelete és onUpdate opciók: - Restrict (alapértelmezés DB-nek megfelelően) — nem engedi a törlést ha van hivatkozás. - Cascade — törlés/ frissítés hatása minden hivatkozásra. - SetNull — beállítja a foreign key-t nullra. - NoAction — DB-n múlik.

Példa:

```

model Post {
  author User @relation(fields: [authorId], references: [id], onDelete: Cascade)
  authorId Int
}

```

4) Migrations — részletesen

Fejlesztési folyamat (gyakori) 1. Módosítod a schema.prisma-t. 2. Futtatod: **npx prisma migrate dev --name describe-change** - Ez létrehozza a migration fájlt és frissíti a helyi DB-t. 3. **npx prisma generate** — (szokásosan a migrate dev lefuttatja) generálja a Prisma Client-et.

Termelés - **npx prisma migrate deploy** — ez a production-ready parancs, CI/CD-ben futtatandó, ahol nincs interaktív fejlesztési lépés. - **prisma migrate resolve** — manuális beavatkozás, ha konfliktus van a migration állapotban. - Drift detection: ha a DB és a migrációk nincsenek szinkronban, drift-et jelezhet a migrációs parancs; ilyenkor introspect/egyeztetés szükséges.

Megjegyzés: **prisma db push** vs **migrate - db push** a sémát offline módon “rátolja” a DB-re migrációs fájlok generálása nélkül — gyors prototípushoz, de nem ajánlott production migrációk helyett, mert nincs verziózott migrációs történet.

5) Prisma Client API — részletes magyarázat

Alap CRUD műveletek (mindegyik Promise-t ad vissza, async/await használatos).

findUnique - Keresés primer kulcs vagy egyedi mező alapján. Ha nincs találat, null-t ad vissza.

```
prisma.user.findUnique({ where: { id: 1 } });
```

findFirst - Az első találatot adja vissza a megadott feltételek alapján. Hasznos compound where és order kombinációknál.

findMany - Lista lekérése; támogatja where, orderBy, take, skip, cursor, include, select.

```
prisma.post.findMany({ where: { published: true }, include: { author: true } });
```

create / createMany - Új rekord létrehozása; createMany tömeges beszúrást tesz lehetővé (egyszerűbb, gyorsabb, de limitáltabb visszajelzés).

update / updateMany - update egy rekordot a where alapján (ha nincs találat, hibát dob). - updateMany több rekordot módosít; visszaad egy számot a módosított rekordokról.

upsert - Update vagy insert egy parancsban (ha nincs létező rekord, létrehozza).

```
prisma.user.upsert({
  where: { email: 'a@b.com' },
  update: { name: 'Updated' },
  create: { email: 'a@b.com', name: 'New' },
});
```

delete / deleteMany - Törlés (delete: egy rekord; deleteMany: több).

aggregate és groupBy - Aggregációk (count, avg, sum, min, max) és groupBy lekérdezések.

Raw queries - `$queryRaw` és `$executeRaw` — nyers SQL futtatása. Vigyázz SQL injection-re: használj parametrizált hívásokat (`$queryRaw` with `Prisma.sql` vagy `? paramok`).

```
await prisma.$queryRaw`SELECT * FROM "User" WHERE email = ${email}`;
```

Include vs Select - include: kapcsolódó modellek teljes objektumait tölti be. - select: kiválasztott mezőket ad vissza (jobb teljesítmény, ha kevesebb mezőt kérsz).

Cursor-based pagination (jobb skálázhatóság) - `cursor`, `take`, `skip` együtt: biztonságosabb nagy adatbázisoknál, mint az offset alapú pagináció.

6) Tranzakciók és atomikus műveletek

transaction(tömb) – Többlekérdezésegyszeretörténő futtatása, vagy elköteleződik, vagy visszagördül. “typesc
`prisma.user.create({ data: ... }), prisma.post.create({ data: ... }));`

Interactive transactions (callback)

- További előnyök: egy tranzakción belül ugyanazt a ``prisma`` példányt használva komplex logika
``typescript

```
await prisma.$transaction(async (tx) => {  
  const user = await tx.user.create({ data: ... });  
  await tx.post.create({ data: { authorId: user.id } });  
});
```

Megjegyzés: Tranzakciók költségesek, csak ahol valóban szükséges (adatkonzisztencia). ACID jelentését lásd a Glossary-ben.

7) Indexek, constraint-ek, composite keys és performance tippek

Indexek - `@@index([field])` vagy `@index` (mező szinten nem mindig elérhető) a népszerű keresési mezőkhöz. - Index nélkül a WHERE szűrések teljes táblaskanolást jelenthetnek.

Unique és kompozit unique - `@unique` mező szinten, `@@unique([a,b])` modell szinten.

Composite primary key - `@@id([a,b])` — ha a rekord azonosítása több mező együttesén alapul.

Performance tippek - Kérj csak annyi mezőt, amennyi szükséges (`select`). - Használj indexeket a gyakori filter mezőknél. - Kerüld a `include: { hugeRelation: true }` hívásokat, ha nem kell minden beágyazott mező. -

Használj cursor-based pagination-öt nagy táblákra. - Ellenőrizd a lekérdezések által generált SQL-t, ha lassúnak tűnik (Prisma logolás lehetősége).

8) Prisma Studio, introspect, db push

Prisma Studio - `npx prisma studio` — grafikus felület, gyors DB böngészéshez, adatellenőrzéshez.

db pull (introspect) - `npx prisma db pull` — meglévő adatbázisból legyártja a schema.prisma modelljeit (reverse engineering). - Hasznos legacy DB-hez, de óvatosan: introspect eredményeit érdemes kézzel áttekinteni.

db push - `npx prisma db push` — schema.prisma alapján az adatbázist frissíti, de nem készít migrációs fájlokat (gyors prototípusokhoz).

9) Seedelés, tesztelés, ephemeral DB-k CI-ben

Seed - package.json-ban `prisma.seed` beállítható, vagy `npx prisma db seed`. - A seed script tipikusan ts-node vagy node scripttel fut.

Tesztelés - CI-ben érdemes ephemeral DB-t indítani (Docker), vagy SQLite in-memory-t használni egyszerűbb integrációs tesztekhez. - Külön adatbázis minden tesztfuttatáshoz gyors izolációt ad (parancs: create database, migrate, run tests).

10) Hibák és kivételkezelés — PrismaClient... hibák

Gyakori kivételek - PrismaClientKnownRequestError - A Prisma által ismert adatbázis hibák (constraint violation, unique violation stb.). Contain code (például P2002 = unique constraint violation). - PrismaClientUnknownRequestError - Ismeretlen DB hiba (nem szabványos). - PrismaClientRustPanicError - Ha a Prisma engine (Rust-ban írt) összeomlik. - PrismaClientInitializationError - Indulási/kapcsolódási hiba (pl. rossz connection string). - PrismaClientValidationError - Hibás használat a Prisma Client API-nál (pl. rossz param).

P2002 példa (unique violation) - Ezt kapod, ha egy unique mezőre duplikált értéket próbálsz menteni.

```
try {
  await prisma.user.create({ data: { email: 'a@b.com' } });
} catch (e) {
  if (e instanceof Prisma.PrismaClientKnownRequestError && e.code === 'P2002') {
    // kezeljük a duplikációt
  }
}
```

Ajánlás: - Készíts központi hibakezelőt (pl. NestJS-ben Exception Filter), hogy ezek kezelése egységes legyen.

11) Security, connection strings, pooling, PgBouncer

Connection string (DATABASE_URL) - Formátum például Postgresnél: `postgresql://user:password@host:5432/dbname?schema=public` - Ne tárold forráskódban; használd környezeti változót és secret manager-t.

Pooling - Prisma kliens általában connection poolingot használ a hatékony DB kapcsolat-kezeléshez. - Postgres esetén gyakori a PgBouncer használata — ez egy connection pooler külső réteg, amely csökkenti a nyitott DB kapcsolatok számát. - Figyelj a `max_connections` limitre a DB-n (Postgres default például 100).

Best practice: - Használd PgBouncer-öt nagy forgalmú env-ekben, vagy szolgáltatói connection pooling megoldást. - Konfiguráld a Prisma connection limitet és a szolgáltatásod thread/worker modelljéhez.

12) Prisma Data Proxy (ha elérhető)

Mi ez? - A Prisma Data Proxy egy hosztolt réteg, amely a kliens és az adatbázis között van, és célja a connection pooling, multi-tenant és serverless scénáriók egyszerűbb kezelése.

Mikor érdemes? - Serverless környezetben (pl. Vercel, AWS Lambda), ahol a rövid életciklus miatt a direkt DB kapcsolat problémás. - Ha nem akarsz saját PgBouncer-t üzemeltetni.

Megjegyzés: Data Proxy szolgáltatói lehetőség, díjazással, és regionális teljesítmény szempontjait is vizsgálj.

13) NestJS + Prisma — integráció

PrismaService (Nest pattern) - Egyszerű singleton service, ami a PrismaClient-et örökli/használja, és kezeli az összekapcsolódást és lekapcsolódást. - Különösen figyelj hot-reload fejlesztésnél (nodemon), mert többszörös PrismaClient példány memory leak-et okozhat.

PrismaService példa (ismétlés)

```
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit, OnModuleDestroy {
  async onModuleInit() { await this.$connect(); }
  async onModuleDestroy() { await this.$disconnect(); }
}
```


Request-scoped problémák - Ha Prisma-t request-scoped szolgáltatásként hozod létre, minden kérésnek új DB kapcsolatot hozhat — ez skálázási gondot jelent. - Javasolt: singleton PrismaService + tranzakciók / tx használata per-request, ha szükséges.

14) Best practices checklist (rövid)

- Használj migrációt (**migrate**) production-ban, ne **db push**.
- Ne tárold a `DATABASE_URL`-t vállalt forráskódban; használj secret management-et.
- Korlátozd a lekérdezett mezőket (select) és kerüld a túl nagy 'include'-okat.
- Figyeld a connection limitet és használj PgBouncer-t ha szükséges.
- Kezeld a Prisma hibakódokat (P2002 stb.) és logold a részleteket.
- Teszt környezet: külön DB vagy ephemeral DB docker konténer.
- Próbáld csökkenteni a N+1 problémákat DataLoader-rel GraphQL esetén.

15) Glossary — minden idegen kifejezés magyarázata (részletes, rövid definíciók)

- ORM (Object-Relational Mapping): olyan szoftverréteg, amely objektumok és relációs adatbázis táblák közötti leképezést biztosít, hogy ne kelljen kézzel SQL-t írni minden művelethez.
- Schema (Prisma): a `schema.prisma` fájl, amely leírja a modelleket (táblákat), mezőket és azok attribútumait, datasource-okat és generátorokat.
- Generator: eszköz-forrás a Prisma-ban, ami kódot generál (pl. Prisma Client).
- Datasource: adatforrás konfiguráció (pl. a DB típus és connection string).
- Migration: adatbázis sémaváltozások verzionált mentése (SQL/DDl fájlok), amiket lépésenként futtatunk.
- Introspect / db pull: meglévő adatbázisból a Prisma sémát visszanyerni (reverse engineering).
- Seed: kezdeti adatok betöltése a fejlesztési vagy teszt DB-be.
- ACID: Atomicity, Consistency, Isolation, Durability — tranzakciós tulajdonságok, amelyek konzisztens DB műveleteket garantálnak.
- Transaction: több DB művelet egyetlen egységként futtatása, amely vagy teljesen sikeres (commit), vagy visszagördül (rollback).
- Atomic: „atomikus” — vagy mindent végrehajtunk, vagy semmit egy tranzakción belül.
- Data Proxy: Prisma által kínált hosztolt proxy réteg a kliens és DB között (pooling/connection menedzsment serverless környezetekhez).
- Connection pooling: a DB kapcsolatok újrahasonosítása felgyorsítja a kapcsolódásokat és csökkenti az open connections számát.
- PgBouncer: Postgres-specifikus connection pooler (külön folyamat), amely megkönnyíti a pool kezelést.

- N+1 probléma: tipikus teljesítményprobléma, amikor egy lekérdezés N rekord után további N lekérdezést indít (példa: 1 lekérdezés a felhasználókra + N lekérdezés mindegyik user posztjaira).
- Cursor-based pagination: paginációs módszer, amely kurzort (egy adott rekord alapján) használ a következő oldalakhoz — robosztusabb nagy adatbázisoknál, mint offset/limit.
- Raw query: nyers SQL lekérdezés Prisma `$queryRaw` vagy `$executeRaw` segítségével.
- `PrismaClientKnownRequestError` (P2002, stb.): Prisma által az adatbázistól visszaadott jól ismert hibák, kódokkal (pl. unique constraint violation).
- Seed script: scriptek, amelyek előre feltöltik a DB-t pl. tesztekhez vagy fejlesztéshez.
- Ephemeral DB: rövid életű adatbázis (pl. CI job indít egy DB konténert teszteléshez, majd törli), izolációs célokra.
- Schema drift: amikor a migrációs történet és az adatbázis aktuális sémája nincs szinkronban.
- Upsert: kombinált update-or-insert művelet (ha van, update; ha nincs, insert).

Végszó és használati javaslatok - Ezt a fájlt másold a repód `docs/Prisma-guide-extended.md` helyére, vagy illeszd be egy meglévő Prisma kézikönyvbe. UTF-8 kódolással mentsd. - Ha szeretnéd, generálok hozzá egy GitHub Actions workflow-ot (pandoc + xelatex), hogy PDF-t készítsen a Markdownból — ugyanazt a megoldást alkalmazhatjuk, mint a NestJS esetében. - További testreszabásokat is szívesen végzek: pl. kiterjesztem a hibajegy- és migrációs esetekre való feljegyzésekkel (konkrét P2002 kezelési minták, migrációs hibák diagnosztikája), vagy átírom a guide-ot könnyebb haladó fejlesztőknek szánt checklistekkel.

(Kérésre elkészítem a PDF-et is és segítek feltölteni a repódba vagy külső hostra. Ha szeretnéd, összeállítom ugyanazt a workflow-t, mint a NestJS esetén, hogy a Markdownból automatikusan PDF készüljön.)